Riphah International University

Advanced Computer Programming

Assignment-3

Name: Shahid Zaman (55123)

Instructor: Mr. M.Usman

Section: BSCS (5-1)

Submission Date: Dec,10, 2025

Assignment report

1. Introduction

This project is named **StudentDB Manager**.
It is a **desktop application** developed using **Java Swing**, **JDBC**, and **MySQL**.

The purpose of this project is to help students understand how:

- Graphical User Interface (GUI) is created in Java
- Java connects with a MySQL database
- Database operations are performed
- Multi-threading keeps the application responsive

---

2. Objective of the Project

The main objective of this project is to design and develop a **Student Database Management System** that:

- Uses **Java Swing** for GUI
- Connects Java with **MySQL** using JDBC
- Performs **Add, View, and Search** operations
- Uses **threading** so the GUI does not freeze during database operations

---

3. Tools and Technologies Used

The following tools and technologies are used:

- **Java SE** – Programming language
- **Java Swing** – For building GUI
- **JDBC (Java Database Connectivity)** – For database connection

- **MySQL Server** – To store student records
- **Thread / SwingWorker** – For background processing
- **NetBeans / IntelliJ / Eclipse** – IDE

---

4. Database Design

Database Name:

```
Student DB
```

Table Name:

```
students
```

Table Structure:

Column Name	Data Type
id	INT (Primary Key, Auto Increment)
first_name	VARCHAR(50)
last_name	VARCHAR(50)
age	INT
email	VARCHAR(100)

The database stores all student records in this table.

---

5. System Design Overview

User Can:

- Add a new student record
- View all students
- Search a student by ID

GUI Components:

- Text fields for First Name, Last Name, Age, Email
- Buttons: Add Student, View Students, Search by ID
- JTable to display student records
- Text area to display status messages

---

6. Functional Description

6.1 Add Student

The user enters student information and clicks the **Add Student** button.
The data is stored in the MySQL database.
A confirmation message is shown in the status area.

6.2 View Students

When the **View Students** button is clicked:

- All student records are fetched from the database
- Records are displayed in a JTable
- Status message shows successful loading

6.3 Search Student by ID

The user enters a student ID and clicks **Search**:

- If the student exists, details are found
- If not, a "Student not found" message appears

---

7. JDBC Implementation

JDBC is used to connect Java with MySQL.
Prepared Statement is used to:

- Prevent SQL injection
- Execute safe and efficient SQL queries

CRUD operations are implemented as:

- **Create** – Add student
- **Read** – View and search students

---

8. Multi-threading Implementation

Database operations may take time and can freeze the GUI.
To solve this problem:

- Database work is done using **Thread / SwingWorker**
- GUI remains responsive
- User can still interact with the application

This improves performance and user experience.

---

9. Exception Handling

Exception handling is used to:

- Handle database errors
- Prevent application crash
- Display error messages in the status box

---

10. Results and Output

- Students can be added successfully
- All student records are displayed in a table
- Student search works correctly
- GUI remains responsive during database work

---

11. Conclusion

In this project, we successfully developed a **Java Swing-based Student Database Management System**.
The project helped us understand:

- GUI design using Swing
- Database connection using JDBC
- CRUD operations
- Multi-threading for responsive UI

12. code

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
```

```
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package studentdbmanager;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.*;

/*
 SIMPLE Student Database Manager
 Swing + JDBC + Basic Thread
*/

public class StudentDBManager extends JFrame {

    // === GUI Components ===
    JTextField tfFN, tfLN, tfAge, tfEmail, tfSearch;
    JTextArea status;
    JTable table;
    DefaultTableModel model;

    // === Database info ===
    String url = "jdbc:mysql://127.0.0.1:3306/shahid";
    String user = "root";
    String pass = "shahid";

    public StudentDBManager() {
```

```
setTitle("StudentDB Manager");
setSize(700, 500);
setDefaultCloseOperation(EXIT_ON_CLOSE);
setLayout(new BorderLayout());

// ---------- TOP PANEL ----------
JPanel top = new JPanel(new GridLayout(5, 2, 5, 5));

tfFN = new JTextField();
tfLN = new JTextField();
tfAge = new JTextField();
tfEmail = new JTextField();

top.add(new JLabel("First Name"));
top.add(tfFN);

top.add(new JLabel("Last Name"));
top.add(tfLN);

top.add(new JLabel("Age"));
top.add(tfAge);

top.add(new JLabel("Email"));
top.add(tfEmail);

JButton addBtn = new JButton("Add Student");
JButton viewBtn = new JButton("View Students");

top.add(addBtn);
```

```
top.add(viewBtn);

add(top, BorderLayout.NORTH);

// ---------- TABLE ----------
model = new DefaultTableModel(
      new String[]{"ID", "First Name", "Last Name", "Age", "Email"}, 0);

table = new JTable(model);
add(new JScrollPane(table), BorderLayout.CENTER);

// ---------- BOTTOM PANEL ----------
JPanel bottom = new JPanel(new BorderLayout());

JPanel searchPanel = new JPanel();
tfSearch = new JTextField(5);
JButton searchBtn = new JButton("Search by ID");

searchPanel.add(new JLabel("ID"));
searchPanel.add(tfSearch);
searchPanel.add(searchBtn);

status = new JTextArea(3, 20);
status.setEditable(false);

bottom.add(searchPanel, BorderLayout.NORTH);
bottom.add(new JScrollPane(status), BorderLayout.CENTER);

add(bottom, BorderLayout.SOUTH);
```

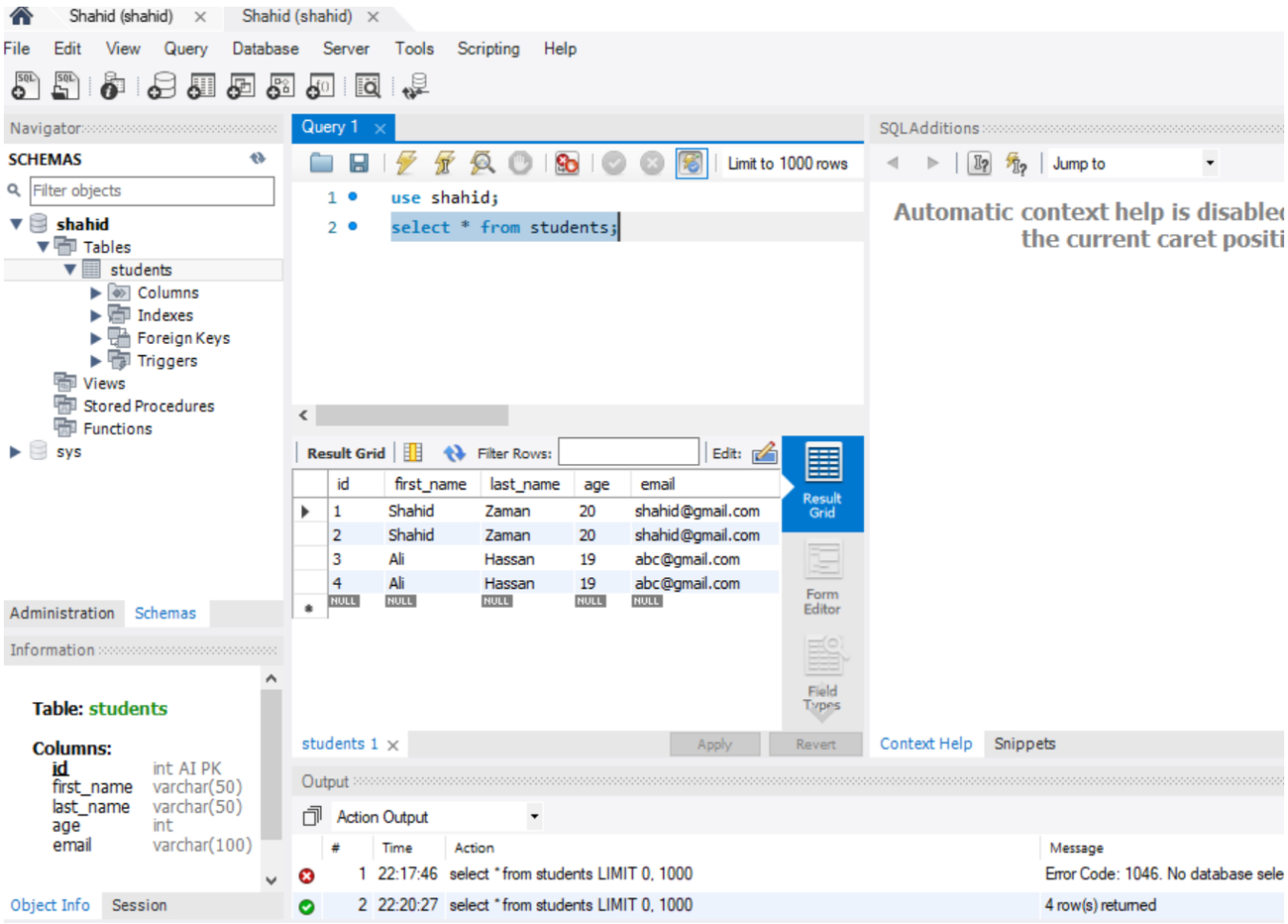
```
    // ---------- BUTTON EVENTS ----------
    addBtn.addActionListener(e -> addStudent());
    viewBtn.addActionListener(e -> viewStudents());
    searchBtn.addActionListener(e -> searchStudent());

    setVisible(true);
}

// ================= ADD STUDENT =================
void addStudent() {

    new Thread(() -> {
        try {
            Connection con = DriverManager.getConnection(url, user, pass);
            String sql = "INSERT INTO students(first_name,last_name,age,email) VALUES (?,?,?,?)";

            PreparedStatement ps = con.prepareStatement(sql);
            ps.setString(1, tfFN.getText());
            ps.setString(2, tfLN.getText());
            ps.setInt(3, Integer.parseInt(tfAge.getText()));
            ps.setString(4, tfEmail.getText());

            ps.executeUpdate();
            status.setText("✅ Student Added Successfully");

            con.close();
        } catch (Exception e) {
            status.setText("❌ Error: " + e.getMessage());
```

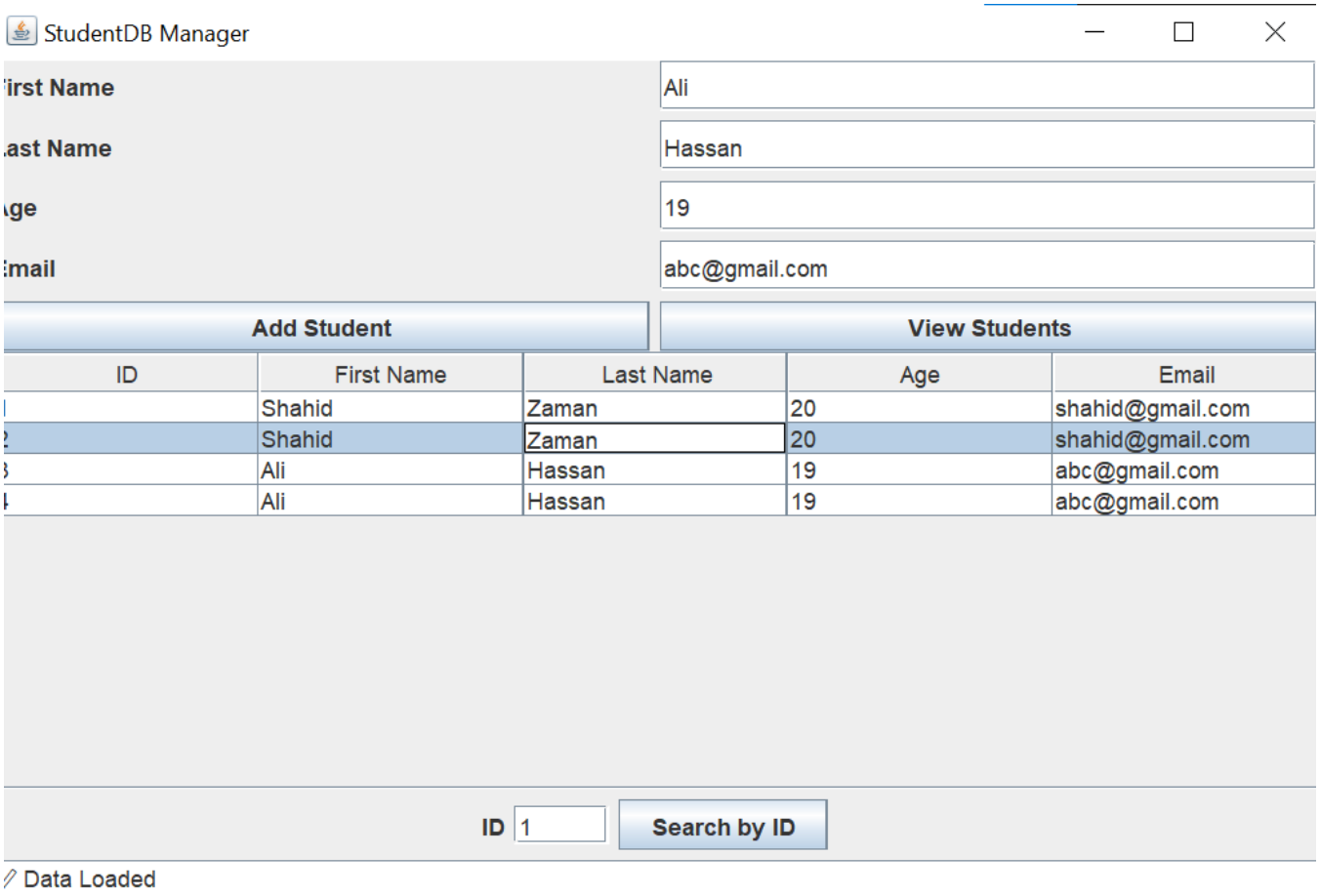
```
        }
    }).start();
}

// ================= VIEW STUDENTS =================
void viewStudents() {

    new Thread(() -> {
        try {
            model.setRowCount(0);

            Connection con = DriverManager.getConnection(url, user, pass);
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM students");

            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getInt("id"),
                    rs.getString("first_name"),
                    rs.getString("last_name"),
                    rs.getInt("age"),
                    rs.getString("email")
                });
            }

            status.setText("✅ Data Loaded");
            con.close();

        } catch (Exception e) {
```

```
            status.setText("❌ Error loading data");
        }
    }).start();
}

// ================= SEARCH STUDENT =================
void searchStudent() {

    new Thread(() -> {
        try {
            int id = Integer.parseInt(tfSearch.getText());

            Connection con = DriverManager.getConnection(url, user, pass);
            String sql = "SELECT * FROM students WHERE id=?";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                status.setText("✅ Found: " +
                        rs.getString("first_name") + " " +
                        rs.getString("last_name"));
            } else {
                status.setText("❌ Student not found");
            }

            con.close();

        } catch (Exception e) {
```

```
            status.setText("❌ Error searching");
        }
    }).start();
  }

  // ================= MAIN METHOD =================
  public static void main(String[] args) {
    new StudentDBManager();
  }
}
```

13. Screenshot





GitHub link:

https://github.com/shahidzaman99/Advance-Programming/tree/main/StudentDBManager